

Evaluación Parcial 2

Libro de Clases Digital

Microservicios, BFF, frontend y despliegue Docker

Problema del Cliente

- El colegio depende de libro de clases físico.
- La consulta de historial estudiantil es lenta.
- La comunicación con apoderados y docentes es limitada.
- Los reportes académicos requieren mucho trabajo manual.

Solución Propuesta

- Plataforma backend basada en microservicios.
- BFF para orquestar flujos del frontend.
- Frontend separado con componentes reutilizables.
- Contenedores Docker para levantar el entorno completo.

Componentes Backend

- autenticacion-seguridad: credenciales, JWT, roles, privilegios y sesiones.
- usuarios-perfiles: información personal, perfiles de estudiante, profesor y apoderado.
- backend-for-frontend: flujo compuesto de registro y comunicación con microservicios.

Patrones de Diseño

- Strategy Pattern: autenticación local extensible a nuevos proveedores.
- Repository Pattern: acceso a datos aislado mediante JPA repositories.
- DTO / Wrapper Pattern: contratos HTTP separados de entidades.
- Filter Chain: JWT, IP, dispositivo y contexto de seguridad.

Patrones Arquitectónicos

- Microservicios con responsabilidad única.
- BFF para reducir complejidad del cliente.
- Arquitectura por capas: controller, service, repository, entity, dto y config.
- JWT con RSA para autenticación stateless.

Seguridad

- Passwords encriptadas con BCrypt.
- Access tokens JWT firmados con RS256.
- Refresh tokens persistidos como hash.
- Sesiones asociadas a dispositivo.
- Blacklist de tokens con Redis.

Usuarios y Perfiles

- Gestiona datos personales y perfiles específicos.
- Usa JPA, validaciones y servicios por dominio.
- Integra RabbitMQ para eventos de creación de usuarios.
- Expone endpoints para completar perfil base y actualizar perfiles.

Frontend Separado

- Proyecto frontend-componentes.
- Incluye package.json y estructura de componente NPM.
- Componentes implementados:
 - auth-card
 - service-summary
 - student-widget

Docker Compose

- Levanta BFF, 2 microservicios y frontend.
- Incluye Redis para autenticación.
- Incluye RabbitMQ para mensajería.
- Usa perfiles dev y H2 en memoria para demostración local.

Puertos

- Frontend: localhost:5173
- BFF: localhost:8082
- Auth: localhost:8081
- Usuarios: localhost:8083
- RabbitMQ Management: localhost:15672

Pruebas y Calidad

- autenticacion-seguridad contiene 46 archivos de prueba.
- Hay pruebas para controladores, DTOs, servicios, repositories, filtros y excepciones.
- Se agregaron .proto mínimos para desbloquear la fase Maven de proto-buf.
- La calidad se apoya en validaciones, transacciones y exception handlers.

Branching

- main: integración estable.
- Ramas por componente: BFF, autenticación, usuarios/perfiles.
- Ramas feature/* para nuevas funcionalidades.
- Integración mediante merges o pull requests.

Cómo Ejecutar

```
docker compose up --build
```

Luego abrir:

```
http://localhost:5173
```

Cierre

La entrega cubre los elementos solicitados por la rúbrica:

- Patrones de diseño.
- Patrones arquitectónicos.
- BFF + 2 microservicios.
- Frontend separado con 3 componentes.
- Dockerfiles y docker-compose.
- Presentación para defensa oral.